

Clinic AI Assistant — Day 2 Post-Mortem Deliverable

Purpose: Document the outcomes of Day 2 implementation, confirm system behavior, capture architectural state, and identify the next logical engineering steps.

Component	Status	Notes
Chat UI	Working	Basic HTML interface renders correctly
Message Input	Working	User input captured and sent to backend
Message History Rendering	Working	Messages displayed in chronological order
POST /chat Endpoint	Working	Frontend successfully communicates with FastAPI backend
CORS Configuration	Working	Browser requests permitted
Static File Serving	Working	FastAPI serves frontend/index.html
Root Route (/)	Working	Loads chat interface
Architecture Alignment	Stable	Frontend, API, and routing working together

Key Observations

Day 2 transitioned the system from a backend-only API to a functioning web application. The frontend is now served directly by FastAPI, ensuring the deployment structure remains consistent between local development and future hosting environments. The request loop between UI and backend is validated and stable.

Recommended Next Step (Day 3)

Introduce the AI service layer. Move response logic from the route controller into `ai_agent.py`, then integrate OpenAI for real response generation. Routes should remain thin controllers while the AI logic lives in the service layer.